

Final Project Proposal

Yihe An

Columbia University
New York, USA
ya2568@columbia.edu

Teresa Co

Columbia University
New York, USA
tc3499@columbia.edu

1 Topic & Description

The midterm evaluations on AI agent systems for software testing by Yihe An served as the inspiration for this research. Software testing requires more than just code creation, according to Yihe's analysis of recent research on LLM-based testing agents. Planning what to test, using tools, refining test cases over multiple iterations, analyzing failures, monitoring coverages, and determining when to quit are all necessary skills for a useful testing agent. We want to keep researching what a testing-oriented AI agent should look like and how effectively it can function in practical situations based on these concepts.

The topic of our final project is to develop and evaluate our own AI agent for repository-level software testing. We plan to design and build an AI agent that takes a software project directory and a testing goal as input, then automatically checks the project, makes a test plan, creates and runs test cases, analyzes coverage and failures, suggests possible bug fixes, and produces a final report. Through this project, we want to explore how such an agent can support the whole testing process, not just single code generation tasks. We want to study this topic because software testing is often boring, repetitive, and time consuming, but it is also very important in software engineering. In theory, this makes testing a good target for AI agents, because many parts of the process could be automated.

We believe this topic is worth studying because, even though recent large language models have made fast progress in code generation and tool use, there is still no widely available end-to-end AI agent that can fully support the software testing process at the repository level. Most current systems only focus on single tasks, such as code completion or single-step test generation, instead of handling the full process from planning to execution, analysis, and reporting. Our project therefore focuses on building a structured AI agent that can coordinate many testing related skills in one workflow.

By building this system ourselves, we hope to study whether a specialized testing agent can provide a more complete, more reproducible, and more practical solution than general purpose coding assistants.

2 Evaluation Plan

We expect to evaluate our system through experiments on small to medium sized open source Python repositories. Since our initial plan is to build a testing agent for Python code, we will focus our evaluation on repositories that use common Python testing frameworks and can support repository-level testing workflows. The evaluation will compare our agent with existing systems and directions discussed in Yihe An's midterm review, especially systems that represent different parts of the testing workflow. In particular, we plan to compare against simpler LLM-based test generation approaches such as AgentCoder [6] and empirical LLM-based unit test generation settings [12], execution-oriented agent designs such as ExecutionAgent [10], and, when appropriate, more specialized multi-agent testing workflows such as HPCAgentTester [11]. These systems represent several qualities found in the evaluation, such as iterative multi-agent refinement, automated test generation, and repository-level test execution, making them pertinent baselines.

The following will be the primary evaluation metrics. The system's ability to properly scan the repository, set up the testing workflow, execute tests, and finish the entire process is known as the execution success rate. This metric is especially important because Yihe's midterm review highlighted reliable repository level execution and reproducibility as key requirements for testing-oriented agent systems. Second, coverage improvement, measured by line coverage and possibly branch coverage before and after agent generated tests. Third, test quality is determined by how well created tests execute, if they contain significant assertions, and whether they steer clear of pointless duplication. Fourth,

the quality of the failure analysis is determined by how well the agent locates possible bug sites or provides a helpful summary of the underlying causes. The fifth metric is efficiency, which is determined by runtime, iteration count, and approximate model usage cost. The review’s conclusion that an effective testing agent must allow generation, execution, iterative refinement, failure localization, traceability, and practical workflow control serves as the impetus for these measurements.

We also plan to include a small case study analysis. For a few representative repositories, we will examine the full agent trace and analyze when the system succeeds or fails. This qualitative evaluation is important because some important behaviors, such as reasonable planning, stopping decisions, iterative improvement, and useful bug summaries, may not be fully captured by coverage alone. This also follows the broader conclusion of the midterm review that software testing needs a more process-driven and structured agent design rather than only one-step generation.

3 Disclosure

Team name: AI-T

Team members: Yihe An (ya2568) & Teresa Co (tc3499).

This project will be completed by a two-person team consisting only of the two of us.

4 Bibliography

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafraan, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, 2023. <https://arxiv.org/abs/2210.03629>
- [2] D. Fu, J. Huang, S. Lu, G. Dong, Y. Wang, K. He, and W. Xu. PreAct: Prediction Enhances Agent’s Planning Ability. arXiv preprint arXiv:2402.11534, 2024. <https://arxiv.org/abs/2402.11534>
- [3] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen. Understanding the Planning of LLM Agents: A Survey. arXiv preprint arXiv:2402.02716, 2024. <https://arxiv.org/abs/2402.02716>
- [4] A. Z. Liu, J. Choi, S. Sohn, Y. Fu, J. Kim, D.-K. Kim, X. Wang, J. Yoo, and H. Lee. SkillAct: Using Skill Abstractions Improves LLM Agents. Proceedings of the ICML 2024 Workshop on LLMs and Cognition, 2024. <https://openreview.net/forum?id=6LG3cIRrF4>
- [5] X. Li, W. Jiao, J. Jin, G. Dong, J. Jin, Y. Wang, H. Wang, Y. Zhu, J.-R. Wen, Y. Lu, and Z. Dou. DeepAgent: A General Reasoning Agent with Scalable Toolsets. arXiv preprint arXiv:2510.21618, 2026. <https://arxiv.org/abs/2510.21618>
- [6] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui. AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation. arXiv preprint arXiv:2312.13010, 2024. <https://arxiv.org/abs/2312.13010>
- [7] B. Liu, X. Li, J. Zhang, J. Wang, T. He, S. Hong, H. Liu, S. Zhang, K. Song, K. Zhu, Y. Cheng, S. Wang, X. Wang, Y. Luo, H. Jin, P. Zhang, O. Liu, J. Chen, H. Zhang, Z. Yu, H. Shi, B. Li, D. Wu, F. Teng, X. Jia, J. Xu, J. Xiang, Y. Lin, T. Liu, T. Liu, Y. Su, H. Sun, G. Berseth, J. Nie, I. Foster, L. Ward, Q. Wu, Y. Gu, M. Zhuge, X. Liang, X. Tang, H. Wang, J. You, C. Wang, J. Pei, Q. Yang, X. Qi, and C. Wu. Advances and Challenges in Foundation Agents: From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems. arXiv preprint arXiv:2504.01990, 2025. <https://arxiv.org/abs/2504.01990>
- [8] B. Han and S. Zhang. Exploring Advanced LLM Multi-Agent Systems Based on Blackboard Architecture. arXiv preprint arXiv:2507.01701, 2025. <https://arxiv.org/abs/2507.01701>
- [9] X. Han and H. Zhu. MASTEST: A LLM-Based Multi-Agent System for RESTful API Tests. arXiv preprint arXiv:2511.18038, 2025. <https://arxiv.org/abs/2511.18038>
- [10] I. Bouzenia and M. Pradel. You Name It, I Run It: An LLM Agent to Execute Tests of Arbitrary Projects. arXiv preprint arXiv:2412.10133, 2025. <https://arxiv.org/abs/2412.10133>
- [11] R. Karanjai, L. Xu, and W. Shi. HPCAgentTester: A Multi-Agent LLM Approach for Enhanced HPC Unit Test Generation. arXiv preprint arXiv:2511.10860, 2025. <https://arxiv.org/abs/2511.10860>
- [12] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. arXiv preprint arXiv:2302.06527, 2023. <https://arxiv.org/abs/2302.06527>
- [13] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyang Qi. MasRouter: Learning to Route LLMs for Multi-Agent Systems. arXiv preprint arXiv:2502.11133, 2025. <https://arxiv.org/abs/2502.11133>